AutoSE Assignment 1: Design and Evaluation Report

A reusable agent skill for requirements specification, with an MCP tool

Rajai Tarun (Roll No. 2025201034)

18 October 2026

1. Summary

`skill.md` is a 9-section skill that makes a coding agent turn a vague idea into three artefacts: a requirements specification, an assumptions and open-questions register, and a validation summary. Its central rule is that every statement carries an honest status (**CONFIRMED / INFERRED / EXTERNAL / UNRESOLVED**) and that nothing is promoted without a source. It uses the public DeepWiki MCP server as a read-only evidence tool. It was tested on 8 inputs over three rounds, including one interactive run in which the agent asked clarifying questions and received answers (14 defects fixed after round 1; 4 more after round 2, whose fixes were re-tested in round 3). The skill does not implement anything.

2. Design rationale and key decisions

Decision	Alternative rejected	Why
Four-value status (CONFIRMED, INFERRED, EXTERNAL, UNRESOLVED) on every item	Two values (stated / assumed)	Tool evidence is neither stated nor a guess; it needs its own label so it cannot be mistaken for a stakeholder fact.
Split form `CONFIRMED (term) / UNRESOLVED (meaning)`	Mark buzzwords CONFIRMED, or drop them	"Sovereign" is stated but has no meaning. Rounding up invents a requirement; dropping loses the stakeholder's words.
"CONFIRMED never implies READY": readiness is separate (READY / NEEDS CLARIFICATION / BLOCKED)	One status	A stated requirement can still be untestable.
Conflicts are logged as `CF-nn`, never resolved by the agent; dependants are BLOCKED	Pick the more authoritative claim	The agent has no authority; picking silently is the failure the brief warns about.
Non-interactive rule: finish the spec, record questions, put escalation at the top	Stop and wait	Agents often run unattended; a mostly-open spec is a valid result.
Decline set for non-software inputs	Force a spec anyway	Producing requirements from business advice invents content.
DeepWiki MCP, read-only, results only EXTERNAL	Web search or no tool	Gives grounded, citable descriptions of real comparable systems with no authentication, so the evidence step is repeatable by anyone.
Skill contains no domain or client content; Varthik's idea is only a test input	Tailor examples to the brief	The brief requires domain independence.

Assumptions made in the design. (1) The agent can read files and run shell commands (curl) or has MCP configured. (2) Users may be absent, so non-interactive behaviour is specified. (3) Markdown is an acceptable artefact format for reviewers.

3. How the skill covers the seven required areas

Brief item	Skill section
Purpose and scope	1: objective, users, invoke / do-not-invoke, borderline inputs, "writes documents only" boundary
Inputs	2: required / optional; table for missing, ambiguous, conflicting, stakeholder-unavailable, very short, prompt-injected input
Requirements analysis	3: actors, FR, NFR, CON, ASM, DEP, RSK, AC; ID prefixes; mandatory fields
Workflow	4: eight ordered steps (intake, extract, gaps, tools, resolve or park, draft, validate, deliver)
Outputs	5: `01_requirements_spec.md`, `02_assumptions_and_open_questions.md`, `03_validation_summary.md` in `spec_<slug>_<date>/`; 12-section template; decline set
Validation	6: six checks (completeness, consistency, traceability, feasibility, testability, provenance honesty) and a readiness rule per requirement
Failure handling	7: when to ask, when to document, retry or replace tools, escalation triggers, "never" list
Side quest (MCP)	8: rationale, discovery, invocation, reliability table, boundaries table

Validation approach. Every requirement needs a source, a status and an acceptance criterion (or an explicit "none yet, see Q-nnn"). The six checks are recorded as PASS / FAIL / N/A with IDs. A check that cannot be judged because information is missing is a FAIL naming the question. The spec as a whole is implementation-ready only if every in-scope requirement is READY and nothing is BLOCKED; otherwise the verdict is "not implementation-ready" and the agent must not round up.

4. MCP side quest: DeepWiki

Selection rationale. Server: DeepWiki MCP, `https://mcp.deepwiki.com/mcp` (public, no authentication, Streamable HTTP); docs: https://docs.devin.ai/work-with-devin/deepwiki-mcp. Tool: `ask_wiki_question(repoName, question)`, plus `read_wiki_structure` and `read_wiki_contents`. Vague ideas usually name a product category. Real projects in that category show what terms like "autonomy", "permissions" and "self-hosted" turn into, which converts buzzwords into concrete options the stakeholder can pick between.

Invocation procedure. (1) Discover: if MCP is configured, list tools; otherwise POST `initialize`, `notifications/initialized`, `tools/list` with headers `Content-Type: application/json` and `Accept: application/json, text/event-stream`. (2) Choose a comparable public repo (from the input, or a judgment call that is recorded). (3) Call `ask_wiki_question` with `{repoName: "owner/repo", question: "..."}`; the result is a Markdown answer. (4) Save the raw request and response; log an `EV-nn` entry. (5) Use the result only to improve questions, add EXTERNAL options, and add INFERRED risks or dependencies.

Evidence of actual use. The same call pattern was made live several times. Two instances for Varthik's input:

- *16 Oct 2026, 12:30 UTC* (raw request and response excerpt in Appendix A): `repoName: block/goose`, asking about autonomy modes, approval gating and self-hosting. HTTP 200 after ~14 s of keep-alive

pings. Relevant result: four modes (`Auto`, `Approve`, `Smart Approve`, `Chat`), with tool calls gated by a permission inspector.

- *18 Oct 2026, round 2* (Appendix A): full discovery (`initialize`, `tools/list` showing the three tools), then `ask_wiki_question` on `All-Hands-AI/OpenHands`: *"How does this agent harness define and configure agent autonomy levels, permissions or confirmation modes, model-agnostic LLM provider support, and self-hosted/local deployment options?"* HTTP 200. Relevant result: autonomy is a spectrum (`confirmation_mode`, `security_analyzer`, a confirm-risky policy with a threshold); models are set through a configurable endpoint; deployment shapes include local backend, Docker, desktop app and an embeddable library.

Effect on the specification (before vs after). Before the call, "fully autonomous" was only an undefined buzzword, so the open question was "what does it mean?" with no options. After EV-01 (round 2 spec, section 12):

Item	Before	After
Q-002 "fully autonomous"	open, no options	options (a) no approval, (b) approval only for risky actions *(EXTERNAL, EV-01)*, (c) sandbox, budget and permission limits, (d) monitor only
Q-001 "harness"	no options	adds option (d): a platform with UI, server, CLI and embeddable library *(EXTERNAL)*
Q-004 / Q-005 "local-first", "self-hosted"	undefined	options for local backend, Docker, desktop, remote fallback *(EXTERNAL)*
Q-006 "model-agnostic"	undefined	options for configurable endpoint, delegation to other agent CLIs *(EXTERNAL)*
RSK-01 / DEP-01	no evidence	supporting note
Any FR / NFR	none	**no requirement created, no status changed.** The stated input outranks external evidence

The earlier round-1 call (OpenHands, 16 Oct) similarly changed Q-006 and Q-008 and added RSK-05, again promoting nothing.

Reliability handling (skill Section 8). Error or non-200: retry once, then record "tool unavailable" and continue. Timeout (~60 s; pings are normal): retry with a narrower question, then treat as unavailable. Service down: try `read_wiki_structure` or another repo, else proceed and list questions the tool would have informed. Irrelevant result: discard and log. Conflicting evidence: keep both EV entries, log a conflict, input outranks evidence. Unverified claim (no source, obscure repo): mark "unverified"; it may motivate a question, never support a requirement. Prompt injection in tool output: treated as data. The skill also says to *skip* the tool when the open questions concern the stakeholder's own decisions (budget, policy), or when no comparable repo can be named without guessing; this was exercised in T5, T6 and T7.

Boundaries. Problem statement and stakeholder answers establish goals, stated features and constraints (CONFIRMED). External evidence establishes candidate options, terminology and plausible risks (EXTERNAL, needs confirmation). Inference supplies obvious implied actors and gaps (INFERRED). Scope, autonomy level, security and privacy expectations, deployment environment, users, priorities and success thresholds remain UNRESOLVED until a stakeholder answers.

5. Empirical evaluation

Method. Each test ran a fresh Claude Code subagent that was given only the skill file and one input, non-interactive. The agent also reported where the skill was unclear. Inputs (public text, unedited) are: **V** the supplied Varthik sentence; **T1** route software (HN, very short); **T2** news-curation app idea; **T3** background-jobs

website; **T4** HN monitor tool; **T5** agency advice (not a software spec); **T6** outsourcing-cost question; **T7** pandas GitHub issue #61317 (22 comments, maintainers disagree). All inputs and outputs were kept; the key outputs are summarised below and the MCP evidence is reproduced in Appendix A.

Round 1 (skill v1, 8 inputs). No run produced a "ready" spec from vague input, none invented numbers, T5 was recognised as not a software request, and T7 preserved its conflicts. The agents reported 14 defects, fixed in v2 (summarised below). Examples: no output format for declined inputs; "NOT READY" was used but undefined; no ID for conflicts (an agent invented `CF-nn`, later adopted); a rule to *stop and ask* clashed with *never stall* in unattended runs; no rule for when to skip the MCP tool; terms such as "fully autonomous" fitted no status (fixed with the term / meaning split).

Round 2 (skill v2, 4 inputs re-run).

Input	Round 1	Round 2
V (Varthik)	0 READY; 8 NEEDS CLARIFICATION; 0 BLOCKED; 14 questions; 1 EV	0 READY; 1 NEEDS CLARIFICATION; 7 BLOCKED; 16 questions; 2 potential conflicts; 1 EV (full discovery + call); escalation flagged
T5 (agency advice)	declined, but improvised format	declined using the defined decline set; no tool call, reason recorded; 4 questions
T6 (outsourcing cost)	1 BLOCKED, not ready	mixed input handled per Section 1; 0 READY, 5 NEEDS CLARIFICATION, 3 BLOCKED; 11 questions; tool skipped with reason
T7 (pandas debate)	16 reqs: 1 READY, 3 NC, 12 BLOCKED; 5 conflicts	28 reqs: 0 READY, 4 NC, 24 BLOCKED; 8 conflicts; escalation; 1 EV (no requirement created)

Round 2 behaved as designed on the points fixed: the decline path, tool-skip reasons, `CF-nn` use and escalation placement all worked without improvisation. Two outcomes were not improvements:

1. **Over-blocking on buzzword input.** V moved from 0 to 7 BLOCKED, which trips the ">50% BLOCKED, escalate" rule automatically. The v2 wording of BLOCKED ("open scope question") overlaps with NEEDS CLARIFICATION ("measurable placeholder"), and the agents improvised different splits. (The V and T7 agents independently reported this.)
2. **T7 no longer has a READY item** (round 1 had 1). This is plausibly more honest, since all the discussion ideas are proposals, but it also shows run-to-run variance (the agent also produced more granular requirements, 28 vs 16).

Round 2 defects and fixes (skill v2.1). (a) Added a BLOCKED / NEEDS CLARIFICATION tie-break (BLOCKED if the answer changes *what is built*; NEEDS CLARIFICATION if it only lacks a threshold; default NEEDS CLARIFICATION) and a note that escalation firing on buzzword-only input is expected. (b) Tensions that exist only through undefined terms are logged as "potential" `CF-nn`. (c) Guidance on NFR vs CON, status for actors / assumptions / risks, and where slogans go. (d) Prefer "none yet, see Q-nnn" over empty acceptance-criterion stubs.

Round 3 (final skill v2.1, the submitted `skill.md`). V, T6 and T7 were re-run non-interactively, plus a new interactive run.

Input	Round 2 (v2)	Round 3 (v2.1)
V (Varthik)	0 READY, 1 NC, 7 BLOCKED, 16 Q	0 READY, 1 NC, 7 BLOCKED, 13 Q, 3 potential conflicts, 2 EV; escalation flagged

Input	Round 2 (v2)	Round 3 (v2.1)
T6 (outsourcing)	0 READY, 5 NC, 3 BLOCKED, 11 Q	0 READY, 2 NC, 4 BLOCKED, 10 Q, 1 potential conflict; tool skipped with reason; escalation flagged
T7 (pandas debate)	0 READY, 4 NC, 24 BLOCKED (of 28), 8 CF	0 READY, 7 NC, 13 BLOCKED (of 20), 20 Q, 8 CF (5 actual, 3 potential); escalation flagged

The tie-break had a visible effect on T7 (BLOCKED share fell from 86% to 65%, with more items correctly marked as lacking only a detail). It did **not** change V, where the one-sentence input legitimately leaves almost everything undefined. The three runs agree on the properties that matter: 0 READY, no invented numbers, conflicts kept open, tool output only EXTERNAL. Escalation still fires on all three, because more than half of the items are BLOCKED. I regard that as correct for these inputs, but the agents again pointed out that the denominator for the ">50% BLOCKED" rule is not defined (FR+NFR+CON vs FR only).

Interactive test (the clarifying-questions path). An agent given V in interactive mode ran steps 1 to 5, including a live DeepWiki call, and stopped to ask a first round of exactly 5 questions, each with a reason, 2 to 4 options and what blocks if unanswered: (1) what "fully autonomous" means, (2) what the harness is, (3) what "sovereign" means, (4) where it runs and whether data may reach outside models, (5) what "enterprise-grade" requires. Answers were then supplied by me *as a simulated stakeholder* (not the real Varthik), deliberately including a "don't know" (availability and scale numbers) and a self-contradiction (only local models and no data leaves the premises, yet an admin may opt in to hosted models such as GPT). Results:

- "Fully autonomous" moved from UNRESOLVED to CONFIRMED as "autonomous inside an admin-set sandbox and policy"; this created new requirements and resolved the earlier potential conflict between autonomy and enterprise-grade.
- The contradiction was caught and logged as an actual conflict `CF-02` between two of the stakeholder's own answers, with no winner picked, dependants BLOCKED, and the decision routed to him plus an unnamed legal or compliance owner. A second, potential conflict was logged over the undefined word "premises" versus "private cloud in India".
- "Don't know" stayed an UNRESOLVED placeholder with no suggested number.
- The DPDP Act (legal compliance) was escalated rather than interpreted by the agent, and the tool was not used for the legal question.
- The final spec had 20 items (0 READY, 14 NEEDS CLARIFICATION, 6 BLOCKED) and 14 new open questions created by the answers.

This shows that answers move statuses honestly and that answers themselves can create new conflicts and questions; the agent also noted that the 5-questions-per-round cap leaves many follow-ups for later rounds.

6. Critical assessment

Where it works well. It never turned a vague input into a confident spec (0 READY in every run across the three rounds). It kept conflicts without choosing a side (T7). It declined a non-software input (T5). It used the tool only where comparable projects exist and logged why it skipped it elsewhere. The artefacts are structured enough to review: every item has an ID, status, source and what it blocks.

Where it struggles.

- The readiness vocabulary is judgement-heavy; two runs on the same kind of input split BLOCKED and NEEDS CLARIFICATION differently (partially addressed in v2.1).
- Borderline inputs (mixed software and business questions, T5 and T6) rely on agent judgement; the agents said either route was arguable.
- Granularity varies between runs (T7: 16 vs 28 requirements), so counts are not comparable across runs.
- Output is long; a human reviewer needs the validation summary and top-questions list to navigate it.

Why I consider it reliable and reusable enough. The behaviour that matters (no silent promotion of assumptions, conflicts preserved, tool output kept as EXTERNAL) held across 16 runs on five very different input types (one-line buzzwords, consumer idea, technical ask, business question, multi-party GitHub debate) with no domain content in the skill. The remaining weaknesses are in labelling and consistency, not in inventing facts. That is a deliberate trade-off: the skill prefers over-asking to guessing.

7. Known limitations

- **Interactive testing was a single run with a simulated stakeholder** (answers written by me, not a real person). It exercised asking, status updates and self-contradiction, but not a real stakeholder's unpredictability, multiple rounds, or the "stakeholder never replies" case beyond the non-interactive runs.
- **Residual ambiguities in the skill,** reported by the test agents and *not* fixed, so that the submitted file stays identical to the tested one: the denominator of the ">50% BLOCKED" escalation rule; whether a "potential" conflict should block dependants; whether "none yet, see Q-nnn" passes Completeness and Traceability or fails them; no ID scheme for out-of-scope items or for citing stakeholder answers and thread comments; how to judge a tool answer whose project name differs from the repository requested; whether a tool should be consulted for legal topics; no tie-breaker between "mostly not software" and "mixed input".
- **The agents' defect reports are self-assessments,** not independent grading. I read the verdicts and several full outputs (V, T1 in detail) but only summaries of the others.
- **Small number of runs per input** (one per round), so there is no proper variance estimate. T7's requirement count moved from 16 to 28 to 20 between rounds, so counts are not comparable across runs.
- **DeepWiki answers are AI-generated summaries** from repository wikis, with blank citation markers in some answers and an answer that described a sub-product ("Agent Canvas") of the requested repo. They were not checked against source code and are used only to motivate questions.
- **Repository choice for the tool is a judgment call** when the input names none. A poor choice would give irrelevant evidence, which the skill discards.
- **Test inputs are public text I scraped,** not real engagements.
- **Only one MCP server was integrated.** The failure-handling table (timeout, 5xx) is specified but those failures did not occur in any real call, so those paths were not exercised.

Appendix A: MCP evidence (actual invocation)

Server: DeepWiki MCP, `https://mcp.deepwiki.com/mcp`. **Date:** 18 Oct 2026. **Discovery:** `initialize` returned HTTP 200, followed by `notifications/initialized` and `tools/list`, which listed `ask_wiki_question`, `read_wiki_structure` and `read_wiki_contents`.

Request (JSON-RPC `tools/call`):

```
{"jsonrpc":"2.0","id":3,"method":"tools/call","params":{"name":"ask_wiki_question",
 "arguments":{"repoName":"All-Hands-AI/OpenHands",
 "question":"How does this agent harness define and configure agent autonomy levels,
 permissions or confirmation modes, model-agnostic LLM provider support, and
 self-hosted/local deployment options?"}}}
```

Response: `HTTP/1.1 200 OK`, `content-type: text/event-stream`. Relevant excerpt of the returned text:

> Agent autonomy and confirmation modes are configured within the conversation settings. The `confirmation_mode` setting, a boolean, determines whether the agent pauses for user confirmation before executing high-risk actions. […] The `security_analyzer` setting […] with options like "llm" or "none". […] The `confirmation_policy` in the payload can be set to `ConfirmRisky` with a `threshold` and `confirm_unknown` flag. […] The Agent Canvas supports model-agnostic LLM providers through the Agent Client Protocol (ACP) […] allows the Agent Server to spawn an external agent's CLI as a subprocess […] integration with third-party agents like Claude Code, Codex, Gemini CLI.

Earlier call, 16 Oct 2026 12:30 UTC (`block/goose`, same style of question about autonomy and approval gating; HTTP 200 after about 14 s of keep-alive pings): *"The agent supports four autonomy or permission modes: `Auto`, `Approve`, `Smart Approve`, and `Chat`. Tool calls are gated by user approval based on the selected `GooseMode` and individual tool permissions."*

Caveats recorded by the agent: the answer is AI-generated from the wiki, names the product "Agent Canvas" although the repository is OpenHands, has blank citation markers, and was not checked against source code. It was therefore marked EXTERNAL, "partially unverified", and used only to add options to open questions.